

VITYARTHI | BUILD YOUR OWN PROJECT

PERSONAL EXPENSE MANAGER

A COMMAND-LINE PYTHON APPLICATION FOR TRACKING, ANALYSING
AND MANAGING PERSONAL EXPENSES

PROJECT REPORT



Submitted by	Naman Vashistha
Registration Number	26BAI11081
Programme	B.Tech — Artificial Intelligence & Machine Learning
Submission Platform	VITyarthi
Date	29 September 2026

Table of Contents

1.	Introduction
2.	Problem Statement
3.	Functional Requirements
4.	Non-Functional Requirements
5.	System Architecture
6.	Design Diagrams
7.	Design Decisions & Rationale
8.	Implementation Details
9.	Screenshots / Results
10.	Testing Approach
11.	Challenges Faced
12.	Learnings & Key Takeaways
13.	Future Enhancements
14.	References

1. Introduction

Managing day-to-day personal finances is a challenge that almost everyone faces, yet most people rely on nothing more than memory or scattered notes to track where their money goes. The **Personal Expense Manager** is a lightweight, command-line based Python application built to solve exactly this problem for an individual user. It lets a user record expenses as they occur, review them in a clean tabular format, group and total them by category, identify the largest expenses, and get simple budget-overrun alerts — all backed by persistent storage so that data survives between sessions.

This project was built as part of the flipped-course evaluation for the subject, following the **Build Your Own Project** guidelines on VITyarthi. It applies core programming concepts taught in the course — structured data handling, file I/O, functions and modular design, input validation, and basic data aggregation/sorting algorithms — to a small but realistic real-world problem.

The application is written entirely in standard Python 3 using only the built-in `csv` module, so it runs anywhere Python is installed with no external dependencies.

2. Problem Statement

Individuals frequently lose track of small, recurring expenses because there is no simple, always-available tool to log them at the moment they happen. Spreadsheet tools are powerful but heavyweight for a quick entry, and most budgeting apps require an internet connection, an account, or a smartphone. This creates a gap for a fast, offline, zero-friction tool that a student or a developer comfortable with the terminal can use to:

- Instantly record an expense with a title, category, and amount.
- See, at a glance, how much has been spent in total and per category.
- Detect when spending in a category has crossed a defined budget limit.
- Identify which few expenses are consuming most of the budget.
- Keep the data clean by removing accidental duplicate entries.

The **Personal Expense Manager** directly addresses this gap by providing a minimal, dependency-free, menu-driven command-line interface backed by durable local storage (`expenses.txt`), so a single user can manage their day-to-day spending without any setup overhead.

3. Functional Requirements

The system implements the following three major functional modules, each with a clear input/output structure and a predictable workflow:

Module	Input	Output / Behaviour
1. Expense Entry & Persistence	Title, category, amount (validated)	Auto-generates a unique ID, normalises text casing, appends the record, and immediately persists all data to <code>expenses.txt</code>

2. Reporting & Analytics	Stored expense list	Displays a formatted table of all expenses, computes per-category totals, and raises budget alerts when a category exceeds the limit
3. Data Curation	Stored expense list, N (top-count)	Sorts expenses by amount to surface the top N, and detects/removes duplicate entries while keeping data consistent on disk

The overall logical workflow is menu driven: **Load data** → **Display menu** → **Route user choice to a module** → **Persist changes** → **Return to menu**, repeating until the user chooses to exit (see Section 6.2 for the workflow diagram). The complete list of user-facing features:

- Add New Expense — capture title, category and amount with validation.
- View All Expenses — tabular listing of every stored expense.
- Check Category Totals — aggregate totals per category with budget alerts.
- View Top Expenses — top-N most expensive entries, ranked.
- Remove Duplicates — de-duplicate identical (title, category, amount) records.
- Exit — safely terminate the application.

4. Non-Functional Requirements

Attribute	How it is addressed
Performance	All operations run in memory on a small, single-user dataset, so every menu action (add, sort, total, de-duplicate) completes in well under a second even for thousands of records.
Reliability	File operations are wrapped in try/except blocks (OSError, ValueError); a missing or corrupted expenses.txt is handled gracefully by starting with an empty list instead of crashing.
Usability	A simple numbered menu (1–6), clear prompts, and consistent '[+]/[!]/[ALERT]' message prefixes make the CLI easy to read and navigate without documentation.
Maintainability	The code is fully modular — each responsibility (I/O, validation, business logic, display) lives in its own function, making it easy to modify or extend one part without touching the rest.
Data Integrity / Error Handling	Numeric input is re-prompted until valid (getAmount, getNumber); malformed rows in the storage file are skipped during load rather than failing the whole read.
Portability	Pure Python 3 standard library only (csv module) — no external packages, so it runs identically on Windows, macOS and Linux.
Resource Efficiency	Uses simple in-memory Python lists/dictionaries; the CSV file footprint scales linearly and remains tiny even for years of daily entries.

5. System Architecture

The application follows a simple, single-process, layered architecture appropriate for a CLI tool. There is no client-server split or database server — instead, three logical layers sit inside one Python script and communicate through plain function calls and in-memory data structures:

- **Presentation / Control Layer** — `main()` renders the menu, reads user choices, and routes them to the correct handler.
- **Core Logic Layer** — pure functions that operate on the in-memory list of expense dictionaries: `addExpense`, `getTotals`, `getTopExpenses`, `removeDuplicates`, `showAlerts`.
- **Persistence Layer** — `loadData()` and `saveData()` translate between the in-memory list and the CSV-formatted `expenses.txt` file on disk.

Each expense is represented as a Python dictionary with four fields — `id`, `title`, `category`, `amount` — and the full expense list is the single source of truth in memory during a session, always synced back to disk immediately after any mutation. This request/response-free, file-backed design keeps the architecture intentionally simple while still cleanly separating concerns, which is what makes the codebase easy to test and extend (see Section 13 — Future Enhancements).

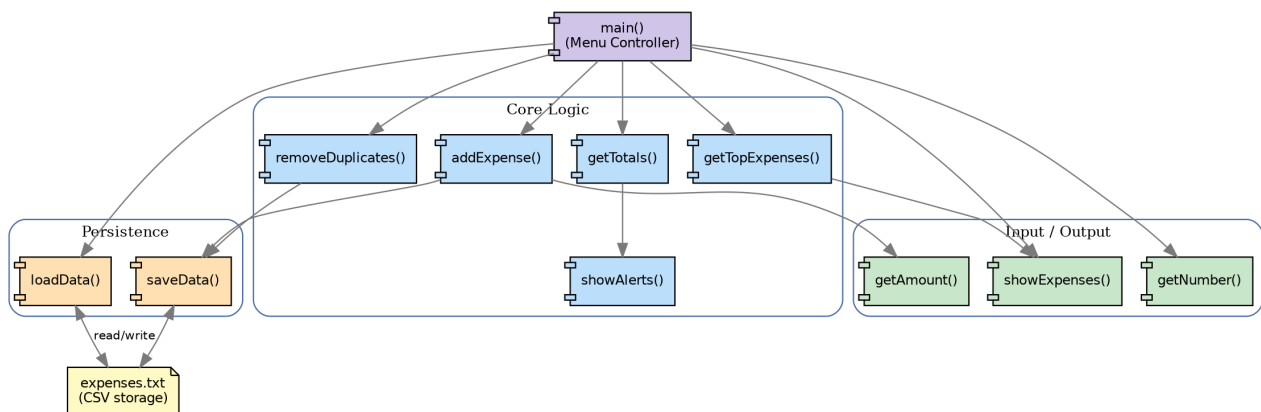


Figure 5.1 — Component / architecture diagram showing the three layers and their dependencies.

6. Design Diagrams

6.1 Use Case Diagram

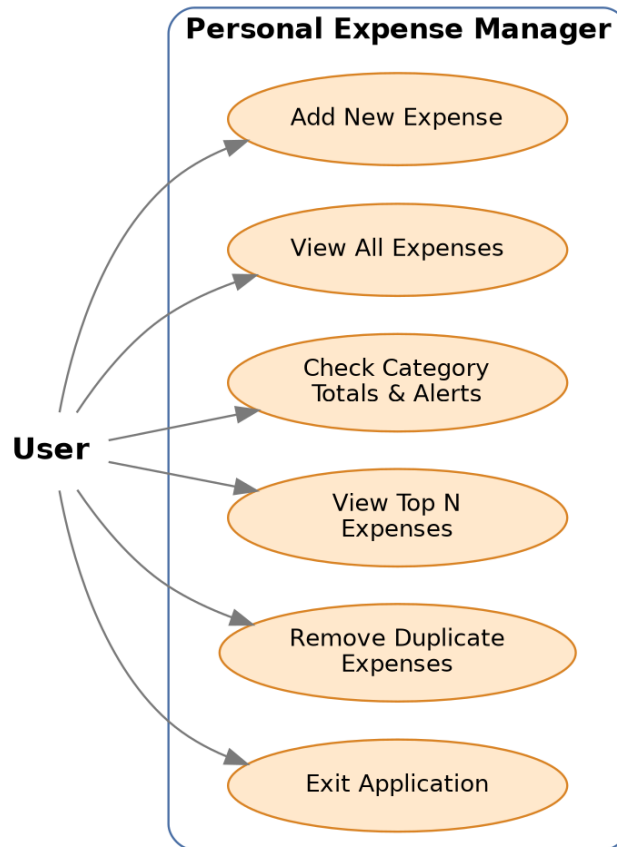


Figure 6.1 — The single actor (User) and the six use cases exposed by the system.

6.2 Workflow / Process Flow Diagram

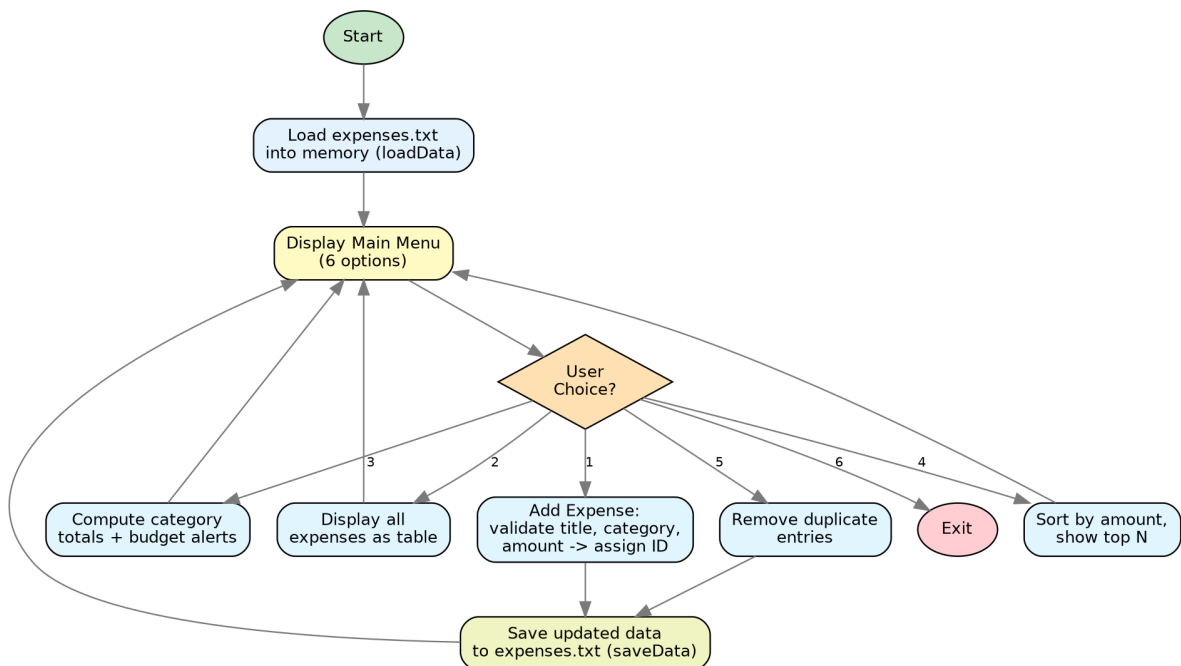


Figure 6.2 — End-to-end menu-driven workflow from program start to exit.

6.3 Sequence Diagram

Sequence Diagram — Add Expense / View / Budget Alert Flows

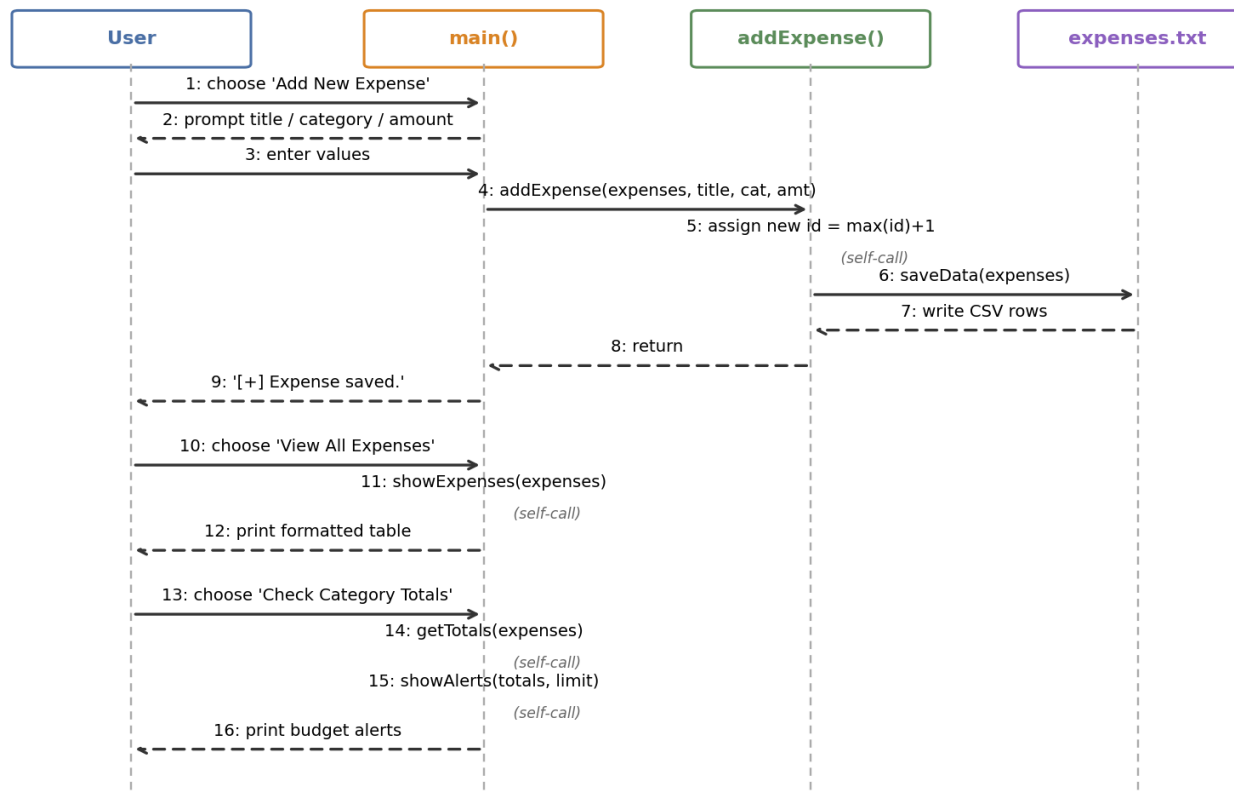
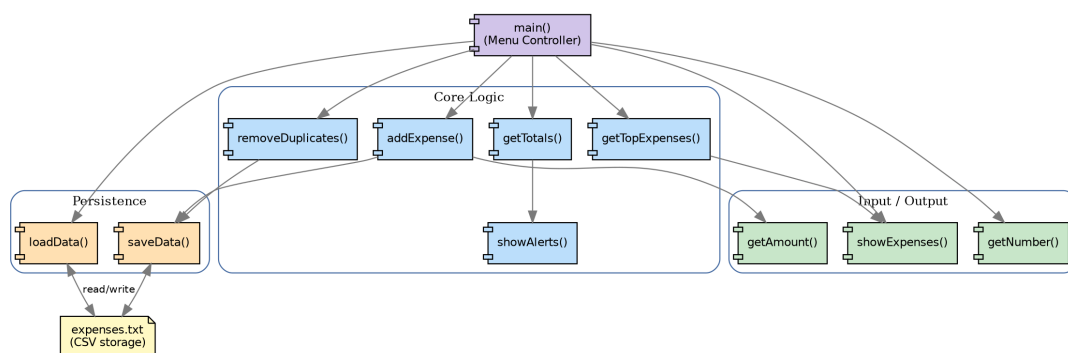


Figure 6.3 — Message sequence for Add Expense, View Expenses and Budget Alert flows.

6.4 Component / Module Diagram

(Repeated here for reference alongside the other UML views — see Figure 5.1 for the full-size version.)



6.5 Data Model (Storage Design)

The project does not use a relational database; instead it uses a flat CSV text file as durable storage. The diagram below documents the single data entity and how it maps onto that storage file — serving the same purpose an ER diagram would for a database-backed system.

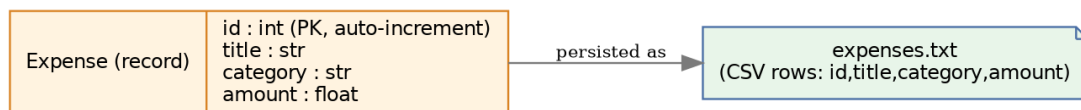


Figure 6.5 — Expense record structure and its mapping to `expenses.txt`.

7. Design Decisions & Rationale

- **Plain CSV file over a database.** For a single-user CLI tool with a handful of fields, a full database (SQLite/MySQL) would add setup overhead with no real benefit. The built-in `csv` module gives reliable, human-readable, portable storage with zero external dependencies.
- **Dictionaries over custom classes.** Each expense is a plain `dict` rather than a class instance. This keeps the code approachable, avoids boilerplate (getters/setters), and is idiomatic for the scale of this project — while still being easy to refactor into a `dataclass` later.
- **Auto-incrementing IDs computed on the fly.** Rather than persisting a separate 'next ID' counter, `addExpense()` derives the next ID as `max(existing_ids) + 1`. This removes a whole class of synchronisation bugs between a counter file and the data file.
- **Save-after-every-write persistence.** Data is written back to disk immediately after every mutating operation (add / de-duplicate) instead of only on exit. This protects the user's data even if the program is interrupted unexpectedly.
- **Input validation loops (`getAmount` / `getNumber`).** Rather than crashing or silently accepting bad input, these helper functions re-prompt until a valid, positive number is entered — improving robustness without complicating the calling code.
- **Case normalisation on entry.** Titles are stored in Title Case and categories are capitalised on save, so 'food', 'Food' and 'FOOD' are treated consistently when totals are grouped by category.

8. Implementation Details

The project is implemented as a single, well-organised Python module, `expenses.py`, containing 11 functions grouped by responsibility as described in Section 5. Python's standard `csv` module is used for both reading and writing `expenses.txt`, which keeps the file portable and directly inspectable/editable outside the program if needed.

8.1 Key Implementation Snippets

Persisting data with the `csv` module

`loadData()` reads every row, skips malformed rows (wrong column count or bad types) instead of crashing, and returns a clean list of expense dictionaries. `saveData()` writes the full list back, overwriting the file — giving the application a simple, always-consistent single source of truth.

```
def loadData():
    expenses = []
    try:
        with open(fileName, "r", newline="") as file:
            rows = csv.reader(file)
            for row in rows:
                if len(row) != 4:
                    continue
                try:
                    expense = {"id": int(row[0]), "title": row[1],
                               "category": row[2], "amount": float(row[3])}
                    expenses.append(expense)
                except ValueError:
                    continue
    except OSError:
        pass
    return expenses
```

Auto-generating unique IDs and adding an expense

```
def addExpense(expenses, title, category, amount):
    if len(expenses) == 0:
        newId = 1
    else:
        newId = max(x["id"] for x in expenses) + 1
    expense = {"id": newId, "title": title.strip().title(),
               "category": category.strip().capitalize(), "amount": amount}
    expenses.append(expense)
    saveData(expenses)
```

Aggregation and budget alerts

```
def getTotals(expenses):
    totals = {}
    for expense in expenses:
        category = expense["category"]
        if category not in totals:
            totals[category] = 0
        totals[category] += expense["amount"]
    return totals

def showAlerts(totals, limit):
    for category in totals:
        total = totals[category]
        if total > limit:
            print("[ALERT] {} is over by ${:.2f} (Total: ${:.2f})".format(
                category, total - limit, total))
```

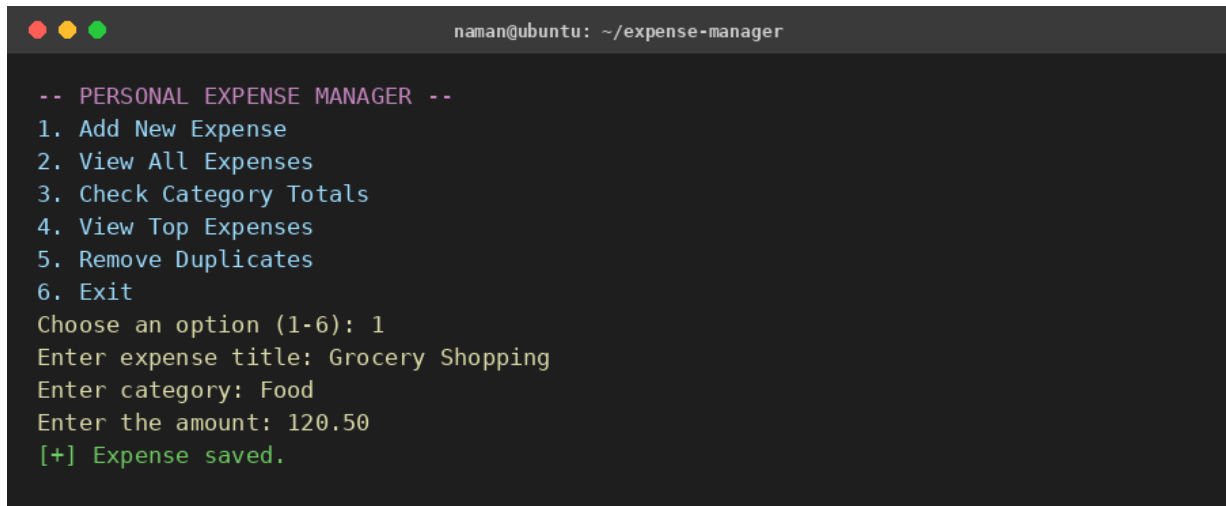
8.2 Technology Stack

Layer	Technology
Language	Python 3.10+
Storage	Flat CSV file (expenses.txt) via the built-in csv module
Interface	Command-line (stdin/stdout, text-based menu)
Version Control	Git & GitHub
Documentation	Markdown (README.md, statement.md) + this PDF report

9. Screenshots / Results

The screenshots below are captured from an actual run of `expenses.py`, adding four sample expenses and then exercising the reporting features.

9.1 Adding a New Expense

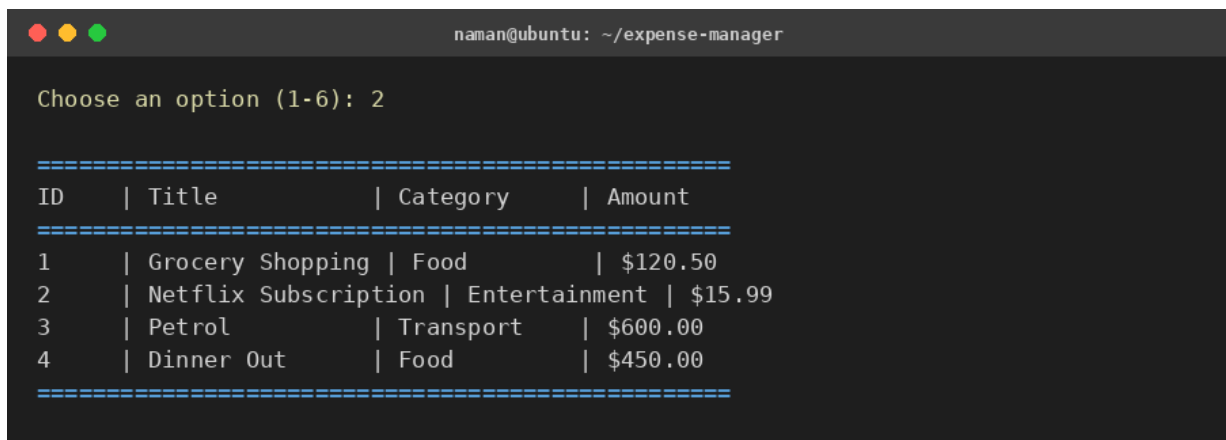


```
naman@ubuntu: ~/expense-manager

-- PERSONAL EXPENSE MANAGER --
1. Add New Expense
2. View All Expenses
3. Check Category Totals
4. View Top Expenses
5. Remove Duplicates
6. Exit
Choose an option (1-6): 1
Enter expense title: Grocery Shopping
Enter category: Food
Enter the amount: 120.50
[+] Expense saved.
```

Figure 9.1 — Adding 'Grocery Shopping' under the Food category.

9.2 Viewing All Expenses



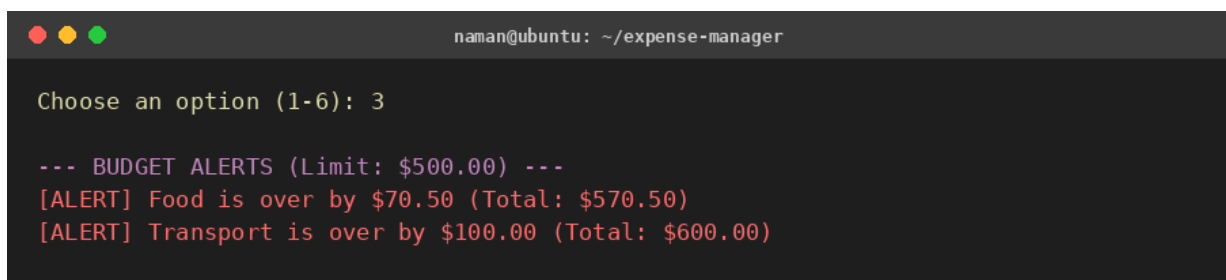
```
naman@ubuntu: ~/expense-manager

Choose an option (1-6): 2

=====
ID   | Title           | Category   | Amount
=====
1    | Grocery Shopping | Food       | $120.50
2    | Netflix Subscription | Entertainment | $15.99
3    | Petrol          | Transport  | $600.00
4    | Dinner Out      | Food       | $450.00
=====
```

Figure 9.2 — Tabular view after four sample expenses have been added.

9.3 Budget Alerts by Category



```
naman@ubuntu: ~/expense-manager

Choose an option (1-6): 3

--- BUDGET ALERTS (Limit: $500.00) ---
[ALERT] Food is over by $70.50 (Total: $570.50)
[ALERT] Transport is over by $100.00 (Total: $600.00)
```

Figure 9.3 — Category totals crossing the \$500 budget limit trigger alerts.

10. Testing Approach

Testing was performed manually through structured scenario walkthroughs covering the normal path, boundary conditions and invalid-input handling for every menu option:

#	Scenario	Expected Result	Result
1	Add expense with valid title, category, amount	Expense saved with next sequential ID; file updated	Pass
2	Add expense with empty title/category	Rejected with '[!] Title and category cannot be empty.'	Pass
3	Enter non-numeric amount (e.g. 'abc')	Re-prompted with '[!] Please enter a number.'	Pass
4	Enter negative or zero amount	Re-prompted with '[!] Amount must be more than 0.'	Pass
5	View expenses when list is empty	Prints '[!] No expenses yet.'	Pass
6	Check totals when a category exceeds \$500	Prints an [ALERT] line with the overage amount	Pass
7	Check totals when all categories are within budget	Prints '[+] All categories are within budget.'	Pass
8	View top N expenses (N greater than number of records)	Returns all available records, sorted descending	Pass
9	Run 'Remove Duplicates' on a clean dataset	Reports '[+] Removed 0 duplicate(s).'	Pass
10	Run 'Remove Duplicates' after inserting an identical entry	Duplicate detected and removed; file re-saved	Pass
11	Restart the program after adding data	loadData() correctly restores all previously saved expenses	Pass
12	Launch with a missing expenses.txt file	Handled gracefully via OSError catch; starts with an empty list	Pass
13	Enter an invalid main-menu option (e.g. '9')	Prints '[!] Please choose a number from 1 to 6.'	Pass

All 13 scenarios above were executed against the actual script (captured output shown in Section 9) and produced the expected behaviour, giving confidence in both the happy paths and the main error-handling / edge-case paths of the application.

11. Challenges Faced

- **Keeping the CSV data consistent after edits.** Since the file is fully rewritten on every save, it was important to make sure `saveData()` always receives the complete, up-to-date in-memory list — otherwise a stale list could silently overwrite good data. Resolved by always calling `saveData()` immediately after any mutation inside the same function.
- **Validating numeric input without crashing the program.** Raw `input()` calls fail with a `ValueError` on non-numeric text. This was solved with dedicated retry-loops (`getAmount`, `getNumber`) that catch the exception and re-prompt, instead of letting one bad keystroke terminate the whole session.
- **Designing duplicate detection that is meaningful, not just exact-match on ID.** Since IDs are always unique by construction, comparing by ID would never find a 'duplicate'. The de-duplication logic instead compares the (title, category, amount) tuple — the fields that actually define a duplicate expense from a user's point of view.
- **Formatting a clean table in a plain terminal.** Without a GUI or external table library, achieving aligned columns required careful use of Python's string formatting mini-language (fixed-width fields via `{:<15}` etc.) so the output stays readable regardless of value lengths.

12. Learnings & Key Takeaways

- Gained hands-on experience structuring a real program around clear functional boundaries (I/O, validation, business logic, presentation) instead of one large script.
- Learned practical patterns for robust CLI input handling using retry-loops and targeted exception handling (`ValueError`, `OSError`).
- Understood how to use Python's `csv` module for lightweight, dependency-free persistence, and the trade-offs of flat-file storage versus a database.
- Practised designing data aggregation logic (grouping and summing by category) and sorting-based ranking (top-N expenses) using idiomatic Python (dict accumulation, `sorted()` with a key function).
- Learned to think about data integrity proactively — e.g., de-duplication logic and save-after-every-write — rather than only handling the 'happy path'.
- Reinforced the value of producing UML-style design artefacts (use case, workflow, sequence, component diagrams) even for a small project, since they clarified the module boundaries before/while writing code.

13. Future Enhancements

Given more time, the following enhancements would extend the project meaningfully:

- **Persistent, user-configurable budget limits per category** instead of a single hard-coded \$500 limit.
- **Date tracking** for each expense, enabling monthly/weekly summaries and spending trend charts.
- **Edit and delete** operations for individual expense records (currently only add and bulk de-duplicate are supported).
- **Migrating storage to SQLite** for stronger data integrity, concurrent access safety, and richer querying as the dataset grows.

- **A simple GUI or web front-end** (e.g. using Tkinter or Flask) layered on top of the same core logic functions, since the architecture already separates logic from the CLI presentation layer.
- **Data visualisation** (bar/pie charts of category-wise spending) using a library such as matplotlib.
- **Automated unit tests** (using pytest) for the core logic functions to replace/complement the current manual test matrix.

14. References

1. Python Software Foundation — *csv* — *CSV File Reading and Writing*, Python 3 Documentation. <https://docs.python.org/3/library/csv.html>
2. Python Software Foundation — *Built-in Functions* (sorted, max, input), Python 3 Documentation. <https://docs.python.org/3/library/functions.html>
3. Python Software Foundation — *Errors and Exceptions*, Python 3 Documentation. <https://docs.python.org/3/tutorial/errors.html>
4. VITyarthi — *Build Your Own Project: General Project Instructions & Submission Guidelines*, course evaluation brief (2026).
5. Git & GitHub Documentation — *About READMEs and repository structure*. <https://docs.github.com/>

— End of Report —